- Building an SQL command or query to send to a database
- Running an operating system command
- Performing a lookup in an LDAP directory
- Sending an HTTP request to another API

If user input is included in any of these cases in an uncontrolled way, the user may be able to influence the command or query to have unintended effects. Such a vulnerability is known as an *injection attack* and is often qualified with the type of code being injected – SQL injection (or SQLi), LDAP injection, and so on.

Header and log injection

There are examples of injection vulnerabilities that do not involve code being executed at all. For example, HTTP headers are lines of text separated by carriage return and new line characters (`"\r\n"` in Java). If you include unvalidated user input in a HTTP header then an attacker may be able to add a "\r\n" character sequence and then inject their own HTTP headers into the response. The same can happen when you include user-controlled data into debug or audit log messages (see chapter 3), allowing an attacker to inject fake log messages into the log file to confuse somebody later attempting to investigate an attack.

The Natter `createSpace` operation is vulnerable to a SQL injection attack because it constructs the command to create the new social space by concatenating user input directly into a string. The result is then sent to the database where it will be interpreted as a SQL command. Because the syntax of the SQL command is a string and the user input is a string, the database has no way to tell the difference.

This confusion is what allows an attacker to gain control. The offending line from the code is the following, which